

MongoDB Indexing Lab

Nuria Arroyo ■ ID: 173605

February 26, 2026

Abstract

This lab analyzes the performance impact of indexing within MongoDB Atlas. By comparing COLLSCAN (collection scan) vs. IXSCAN (index scan), we quantify efficiency gains in query execution time and resource utilization.

1 Environment Setup

- ✓ **Platform:** MongoDB Atlas + Compass
- ✓ **Database:** tp1
- ✓ **Collection:** restaurantes

2 Procedure (Assignment Instructions)

1. Execute queries **without indexes** and observe performance using `explain`.
2. Create the specified indexes (Section 3.5).
3. View / verify indexes.
4. Execute the same queries **with indexes** and observe changes.
5. Compare and analyze results.
6. Delete indexes and re-check query performance.

3 Step 1 — Baseline: queries without indexes

Before making the query we verify the existing indexes on the collection.

Command: Verify index state (baseline)

```
1 // MongoDB shell / mongosh
2 db.restaurantes.getIndexes()
```

Output:

```
1 // MongoDB shell / mongosh
2 { v: 2, key: { _id: 1 }, name: '_id_' }
```

No index beside the default `_id_` index exists at this point.

Now we run the query to find all Italian restaurants and explain it with execution stats.

Command: query all Italian restaurants with explain

```
1 // MongoDB shell / mongosh
2 db.restaurantes.find({ cuisine: "Italian" }).explain("executionStats")
```

Here we may observe the full explain output.

Output: Paste your output here

```
1 // MongoDB shell / mongosh
2 {
3   explainVersion: '1',
4   queryPlanner: {
5     namespace: 'tp1.restaurantes',
6     parsedQuery: {
7       cuisine: {
8         '$eq': 'Italian'
9       }
10    },
11   indexFilterSet: false,
12   queryHash: '98FD4995',
13   planCacheShapeHash: '98FD4995',
14   planCacheKey: '1DC1C771',
15   optimizationTimeMillis: 0,
16   maxIndexedOrSolutionsReached: false,
17   maxIndexedAndSolutionsReached: false,
18   maxScansToExplodeReached: false,
19   prunedSimilarIndexes: false,
20   winningPlan: {
21     isCached: false,
22     stage: 'COLLSCAN',
```

```

23     filter: {
24       cuisine: {
25         '$eq': 'Italian'
26       }
27     },
28     direction: 'forward'
29   },
30   rejectedPlans: []
31 },
32   executionStats: {
33     executionSuccess: true,
34     nReturned: 1070,
35     executionTimeMillis: 26,
36     totalKeysExamined: 0,
37     totalDocsExamined: 25356,
38     executionStages: {
39       isCached: false,
40       stage: 'COLLSCAN',
41       filter: {
42         cuisine: {
43           '$eq': 'Italian'
44         }
45       },
46       nReturned: 1070,
47       executionTimeMillisEstimate: 21,
48       works: 25357,
49       advanced: 1070,
50       needTime: 24286,
51       needYield: 0,
52       saveState: 1,
53       restoreState: 1,
54       isEOF: 1,
55       direction: 'forward',
56       docsExamined: 25356
57     }
58   },
59   ok: 1
60 }

```

3.1 Execution Results

Metric	Value
Winning Plan Stage	COLLSCAN
Execution Time (ms)	26
Docs Examined	25356

3.2 Baseline notes

From the explain output, record:

- Winning plan stage (expected: COLLSCAN without an index on `cuisine`)

- nReturned
- totalDocsExamined
- totalKeysExamined
- executionTimeMillis

Baseline summary (fill in).

Winning plan: COLLSCAN

nReturned: 1070 Docs examined: 25356

Keys examined: 0 Time (ms): 26

4 Step 2 — Create indexes (Section 3.5)

4.1 Q20: increasing index on cuisine

Command: Create single-field index on cuisine

```
1 // MongoDB shell / mongosh
2 db.restaurantes.createIndex({ cuisine: 1 })
```

Output:

```
1 // MongoDB shell / mongosh
2 cuisine_1
```

4.2 Q21: compound index (cuisine asc, address.zipcode desc)

Command: Create compound index on cuisine and address.zipcode

```
1 // MongoDB shell / mongosh
2 db.restaurantes.createIndex({ cuisine: 1, "address.zipcode": -1 })
```

Output:

```
1 // MongoDB shell / mongosh
2 cuisine_1_address.zipcode_-1
```

5 Step 3 — View indexes

Command: Verify indexes created

```
1 // MongoDB shell / mongosh
2 db.restaurantes.getIndexes()
```

Output:

```
1 // MongoDB shell / mongosh
2 [
3   { v: 2, key: { _id: 1 }, name: '_id_' },
4   { v: 2, key: { cuisine: 1 }, name: 'cuisine_1' },
5   {
6     v: 2,
7     key: { cuisine: 1, 'address.zipcode': -1 },
8     name: 'cuisine_1_address.zipcode_-1'
9   }
10 ]
```

```
9   }
10  ]
```

6 Step 4 — Re-run queries with indexes

6.1 Italian restaurants query (with indexes)

Command: Explain with execution stats (indexed)

```
1 // MongoDB shell / mongosh
2 db.restaurantes.find({ cuisine: "Italian" }).explain("executionStats")
```

Output:

```
1 // MongoDB shell / mongosh
2 {
3   explainVersion: '1',
4   queryPlanner: {
5     winningPlan: {
6       stage: 'FETCH',
7       inputStage: {
8         stage: 'IXSCAN',
9         indexName: 'cuisine_1'
10      }
11    }
12  },
13  executionStats: {
14    executionSuccess: true,
15    nReturned: 1070,
16    executionTimeMillis: 12,
17    totalKeysExamined: 1070,
18    totalDocsExamined: 1070
19  },
20  ok: 1
21 }
```

Indexed summary (fill in).

Winning plan: FETCH (IXSCAN on cuisine_1)

nReturned: 1070 Docs examined: 1070

Keys examined: 1070 Time (ms): 12

7 Step 5 — Analysis

7.1 Comparison table

Metric	Baseline (no index)	With index
Winning plan stage	COLLSCAN	FETCH → IXSCAN (cuisine_1)
nReturned	1070	1070
Docs examined	25356	1070
Keys examined	0	1070
Execution time (ms)	26	12

7.2 Discussion

The baseline query execution without an index resulted in a COLLSCAN winning plan, meaning the database performed a full collection scan. In this scenario, MongoDB examined all 25,356 documents in the collection in order to return 1,070 matching documents. No index keys were examined, and the execution time was 26 ms.

After creating the index on `cuisine`, the winning plan changed to IXSCAN (with a FETCH stage). This indicates that MongoDB used the index to directly locate matching documents instead of scanning the entire collection. The number of examined documents decreased dramatically from 25,356 to 1,070, exactly matching the number of returned documents. Additionally, 1,070 index keys were examined, and execution time decreased to 12 ms.

This demonstrates a significant improvement in query efficiency. The index reduced unnecessary document scans, lowered resource utilization, and improved response time. The change from COLLSCAN to IXSCAN confirms that indexing optimizes selective queries by narrowing the search space to only relevant entries.

8 Step 6 — Delete indexes and re-check

8.1 Drop created indexes

Drop indexes (replace names with your actual index names)

```
1 // MongoDB shell / mongosh
2 db.restaurantes.dropIndex("cuisine_1")
3 db.restaurantes.dropIndex("cuisine_1_address.zipcode_-1")
```

Output:

```
1 // MongoDB shell / mongosh
2 {
3   nIndexesWas: 2,
```

```
4   ok: 1
5 }
```

8.2 Explain again after dropping

Explain after dropping indexes

```
1 // MongoDB shell / mongosh
2 db.restaurantes.find({ cuisine: "Italian" }).explain("executionStats")
```

Paste explain output

```
1 // MongoDB shell / mongosh
2 {
3   queryPlanner: { winningPlan: { stage: 'COLLSCAN' } },
4   executionStats: {
5     nReturned: 1070,
6     executionTimeMillis: 9,
7     totalKeysExamined: 0,
8     totalDocsExamined: 25356
9   },
10   ok: 1
11 }
```

8.3 Post-drop discussion

Overall, the experiment clearly demonstrates the impact of indexing on query performance in MongoDB. Without an index, the database performs a full collection scan, examining every document regardless of relevance. With an index, the query planner switches to an index scan, drastically reducing the number of examined documents and improving efficiency. Removing the index restores the original full-scan behavior. Therefore, proper index design is essential for optimizing query performance, especially in large datasets.